Project Report.

URL Shortener

SUBMITTED BY

Name: Anant Dixit

Reg No: 25BCE10980

Course: Fundamentals in AI and ML

TITLE -: URL Shortener

INTRODUCTION

URL Shortener is a command-line tool.

A URL shortener converts an URL into a short code so it becomes easier to share and remember. The URL Shortener project is a command-line version that runs in the terminal and stores all mappings in 'urls.json'.

MOTIVATION OF THE PROJECT

- URLs are hard to type and read.

- A command-line tool is lightweight. Does not need a browser or graphical user interface.

PROBLEM STATEMENT

- Long URLs are inconvenient to share and remember.

- Manual copying. Remembering them is error-prone.

OBJECTIVE OF THE PROJECT

Build a terminal- URL shortener that:

- Accepts ‘shorten <URL>’ ‘expand <CODE>’ ‘list’ ‘help’ ‘quit’.

- Generates a unique short code for each URL.

- Stores all mappings in ‘urls.json’ for persistence.

EXISTING METHODS

- Web-based shorteners.

- Online services that provide graphical user interface dashboards and detailed analytics.

PROS AND CONS OF THE STATED METHODS

- Pros: The advantages of the system include its ability to deliver analytics and its capabilities for branding purposes and its implementation of password security.

- Cons: The system requires an Internet connection and it needs a graphical user interface but it consumes too many resources to function in basic command-line environments.

HARDWARE AND SOFTWARE REQUIREMENTS

- Hardware: Any PC/laptop with Python.

- Software: Python 3.x

A text editor: Terminal

METHODOLOGY AND GOAL

- Read existing URLs from ‘urls.json’.

- For ‘shorten <URL>’:

Generate a 6-character code if not already used.

Store ‘code → URL’ in a dictionary and save to ‘urls.json’.

- For ‘expand <CODE>’: look up the code. Print the URL.

- For ‘list’: print all stored codes and URLs.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

- ‘load_db()’ / ‘save()’: JSON read/write.

- ‘Shorten(sv, url)’:. Stores a short code.

- ‘Expand(sv, code)’: retrieves the original URL.

- ‘Main()’: command-line loop that parses user commands.

ALGORITHM DEVELOPMENT

1. Read ‘urls.json’ into a dictionary.

2. For ‘shorten’:

If URL already exists, reuse its code.

Else generate a random code and store it.

3. For ‘expand’:

Check if code exists; print URL or "not found”.

4. For ‘list’: Print all entries.

SOFTWARE ARCHITECTURAL DIAGRAM

Terminal (User input)

↓

URL Shortener CLI (Python)

↓

Data Layer (urls.json – code → URL mappings)

CODING

- Use ‘json.load’ / ‘json.dump’ to read and write ‘urls.json’.

- Use ‘random.choices(string.ascii_letters + string.digits)’ for 6-character codes.

- Use a ‘while ‘ loop with ‘input()’ to accept commands.

```python
import json
import random
import string
from pathlib import Path

file_name = "urls.json"

def load_data():
    path = Path(file_name)

    if path.exists():
        try:
            f = open(file_name, "r")
            data = json.load(f)
            f.close()
            return data
        except:
            print("file issue... starting fresh")
            return {}
    else:
        return {}

def save(sv):
    try:
        f = open(file_name , "w")
        json.dump(sv,f,indent = 2)
        f.close()
    except:
        print("Error while saving")

def code_gen():
    chars = string.ascii_letters + string.digits
    code = ""
    i = 0

    while(i < 6):
        code = code + random.choice(chars)
```

```python
def code_gen():
    while(i < 6):
        code = code + random.choice(chars)
        i += 1

    return code
def shorten(sv,long_URL):
    for c in sv:
        if sv[c] == long_URL:
            return c

    code = code_gen()

    while True:
        if code not in sv:
            break
        else:
            code = code_gen()

    sv[code] = long_URL

    save(sv)

    return code

def expand(sv,code):
    if code in sv:
        return sv[code]
    return None

def main():
    print("URL Shortener")
    print("------------------------")
    print("commands : shorten, expand, list, help, quit\n")

    sv = load_data()
```

```python
def main():
    sv = load_data()

    while True:
        user_input = input("> ")

        if user_input == "":
            continue                #Ignoring Blank input by the user
        parts = user_input.strip().split(" ")
        command = parts[0].lower()

        if command == "quit":
            print("closing program...")
            break

        elif command == "help":
            print("shorten <url>")
            print("expand <code>")
            print("list")
            print("quit")

        elif command == "list":
            if len(sv) == 0:
                print("no urls yet")
            else:
                print("saved urls:")
                for k in sv:
                    print(k + " -> " + sv[k])

        elif command == "shorten":
            if len(parts) < 2:
                print("Enter url pls")
                continue

            url = ""
            for i in range(1,len(parts)):
```

```python
def main():
            url = ""
            for i in range(1,len(parts)):
                url += parts[i]
                if i != len(parts) - 1:
                    url += " "

            code = shorten(sv,url)

            print("short code is: ",code)
            print("use:", "http://short.com/" + code)

        elif command == "expand":
            if len(parts) <2:
                print("enter code")
                continue

            code = parts[1]

            result = expand(sv,code)

            if result != None:
                print("Original: ",result)
            else:
                print("not found")

        else:
            print("unknown command type(help)")

if __name__ == "__main__":
    main()
```

OUTPUT

> shorten https://example.com/very/long/path

Short code: abc123

Example: http://short.com/abc123

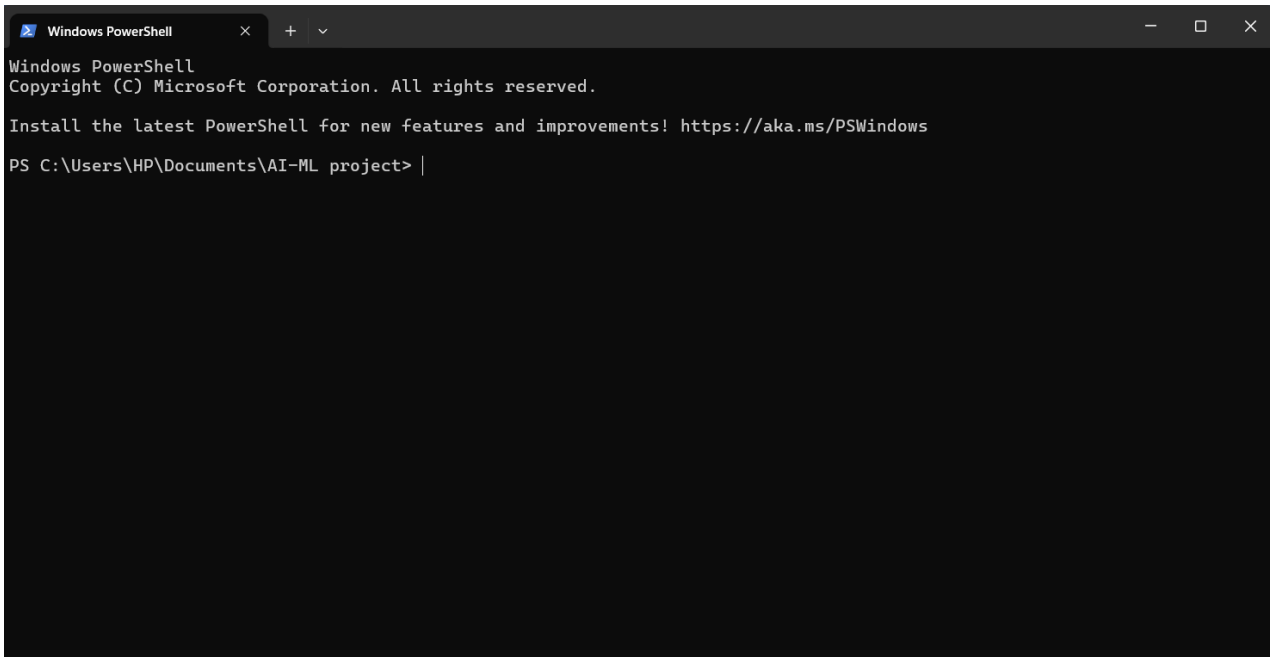
> expand abc123

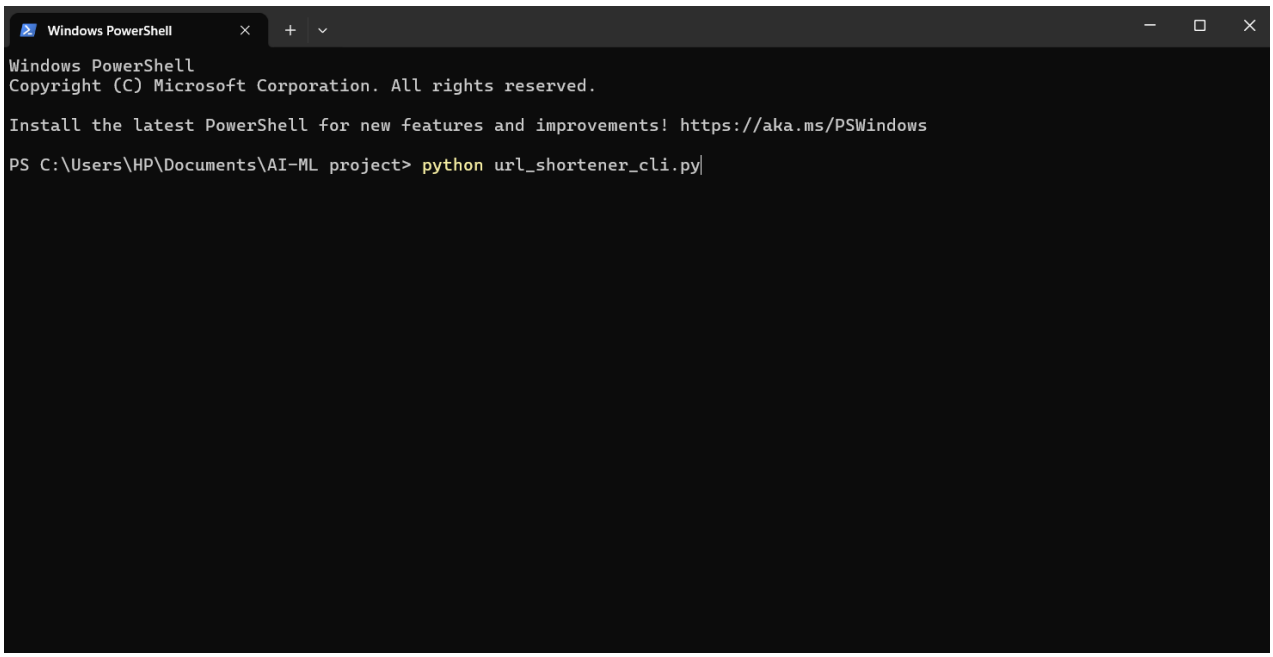
URL: https://example.com/very/long/path

> list

Stored URLs:

abc123 -> https://example.com/very/long/path





```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HP\Documents\AI-ML project> python url_shortener_cli.py
URL Shortener
------------------------
commands : shorten, expand, list, help, quit

>
```

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HP\Documents\AI-ML project> python url_shortener_cli.py
URL Shortener
------------------------
commands : shorten, expand, list, help, quit

> shorten https://www.hackerrank.com/
short code is:  BrKTBN
use: http://short.com/BrKTBN
> expand BrKTBN
Original:  https://www.hackerrank.com/
> help
shorten <url>
expand <code>
list
quit
> list
saved urls:
BrKTBN -> https://www.hackerrank.com/
> quit
closing program...
PS C:\Users\HP\Documents\AI-ML project>
```

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- Pure command-line no graphical user interface.

- JSON-based persistent storage.

- Simple, code.

PROJECT OUTCOMES

- Built a working usable URL shortener.

- Learned file-handling random code generation and terminal-based input loops.

TESTING AND REFINEMENT

- Tested: shortening, expanding, listing invalid commands.

- Refined: prompts, clear error messages and smooth flow.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- tool for shortening links quickly.

- Lightweight internal tool where graphical user interface tools are not allowed.

- Base for later adding analytics.

CONTRIBUTION/FINDINGS OF THE PROJECT

- Created a practical tool that improves URL manageability.

- The research showed that basic command-line tools provide significant advantages to users when implementing their work processes

LIMITATION/CONSTRAINTS OF THE SYSTEM

- The system currently lacks any integrated analytical functions.

- No spam / URL detection.

- No advanced features like password protection or expiry.

CONCLUSIONS

- A command-line-based URL shortener is a efficient way to manage long links.

- The project demonstrates file handling, modular design and terminal interaction.

FUTURE ENHANCEMENTS

- Add analytics.

- Add spam / URL detection rules.

- Add password protection or expiry for codes.

- Allow importing/exporting URL mappings.

REFERENCES

- Python ‘json’ module documentation.

- Python module documentation.